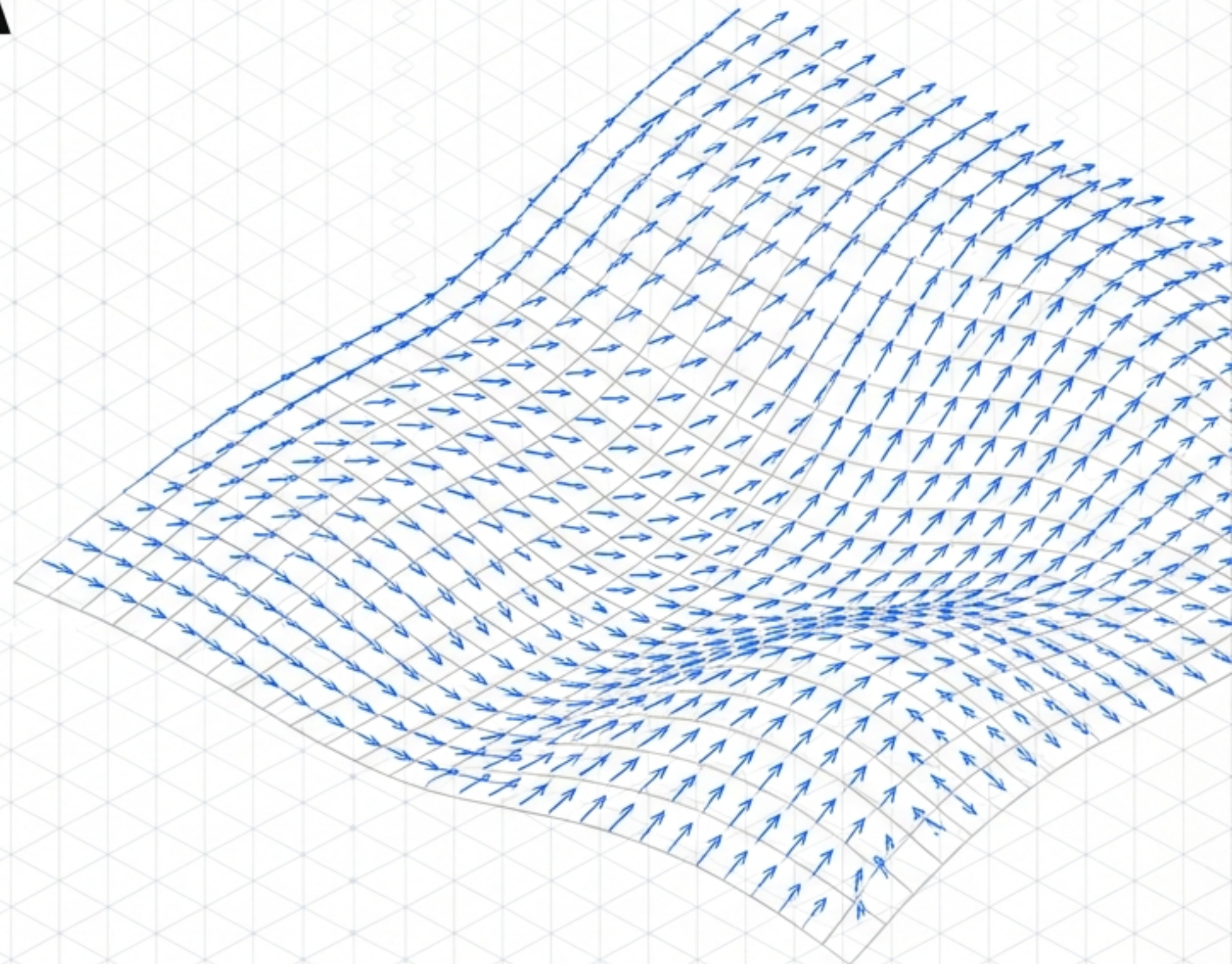


LINEAR ALGEBRA FOR MACHINE LEARNING

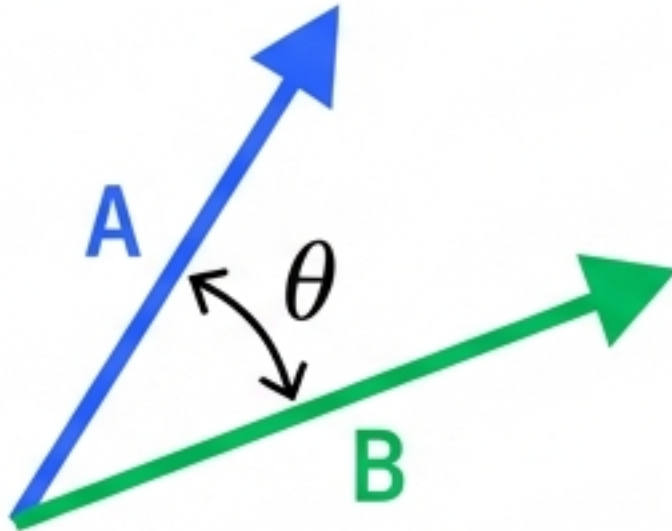
Vectors, Geometry, and
Matrix Operations



The Architecture of Data Science

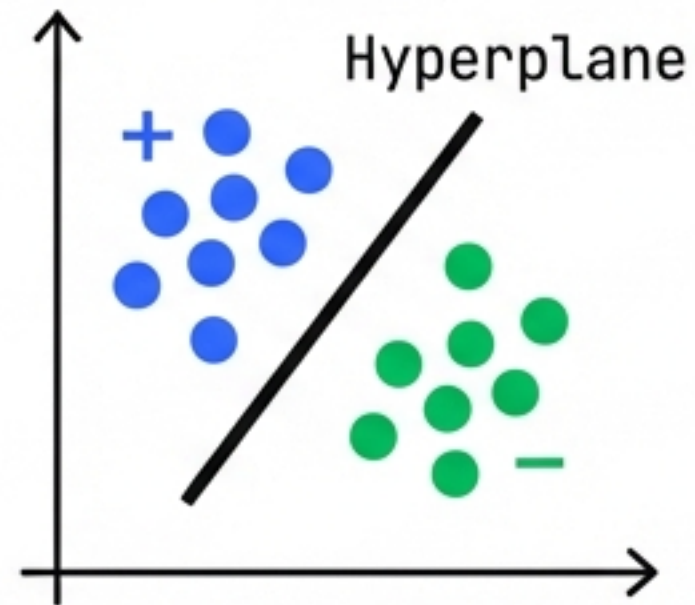
To understand how algorithms 'think', we must understand how they measure, separate, and transform data.

01. RELATIONSHIPS



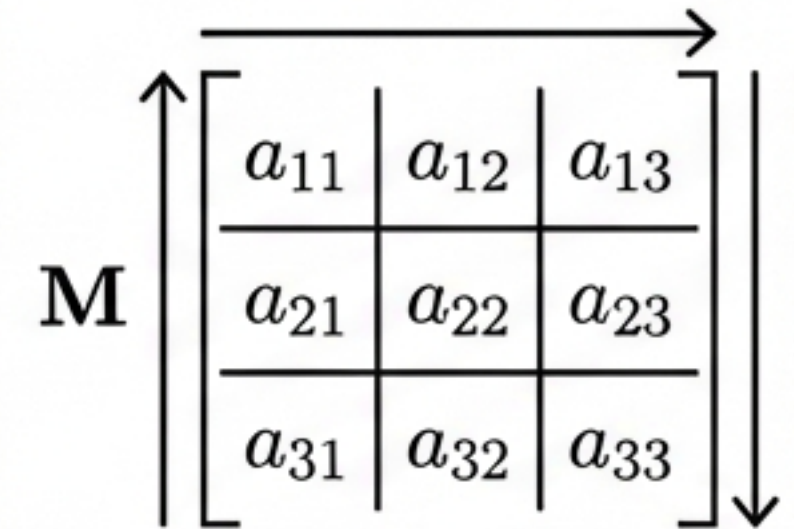
Similarity
Projections

02. BOUNDARIES



Hyperplanes
SVMs

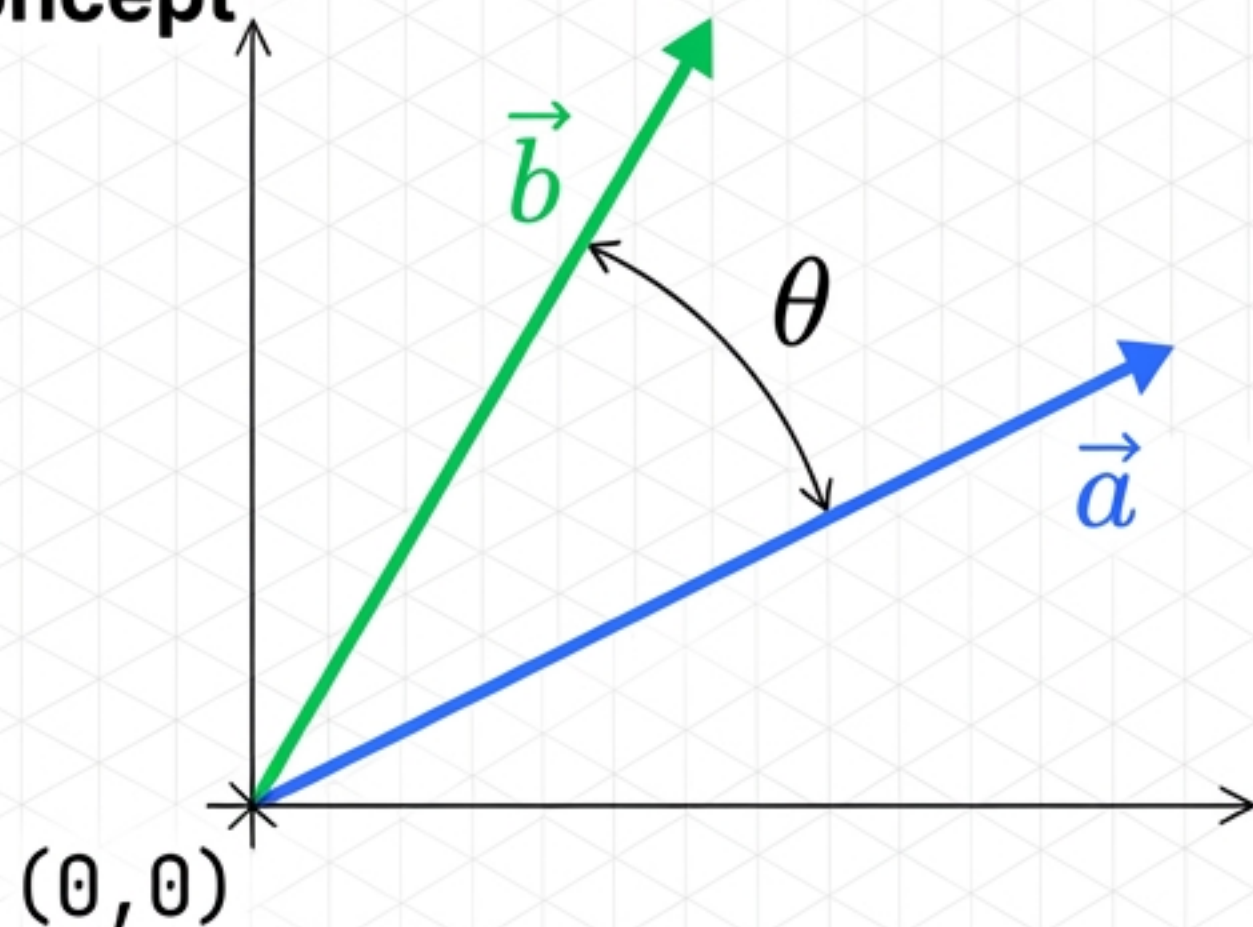
03. ENGINES



Transformations
Rank

Measuring Alignment: The Angle Between Vectors

Concept



Same Direction: 0°
Perpendicular: 90°
Opposite Direction: 180°

Math & Application

$$\cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{a}|| ||\mathbf{b}||}$$

ML Applications

- **Cosine Similarity:** Measuring text similarity in NLP.
- **PCA:** Variance determined by eigenvector orientation.
- **SVM:** Classification via angles to the hyperplane.

Case Study: Calculating the Angle

Scenario: vector **a** = (1, 2), vector **b** = (3, 4)

The Math

1. Dot Product:

$$(1)(3) + (2)(4) = 3 + 8 = 11$$

2. Magnitudes:

$$\|\mathbf{a}\| = \sqrt{1^2 + 2^2} = \sqrt{5} \approx 2.24,$$

$$\|\mathbf{b}\| = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$$

3. Formula:

$$\cos \theta = \frac{11}{5\sqrt{5}} \approx 0.984$$

4. Result: $\theta \approx 16.26^\circ$

The Code

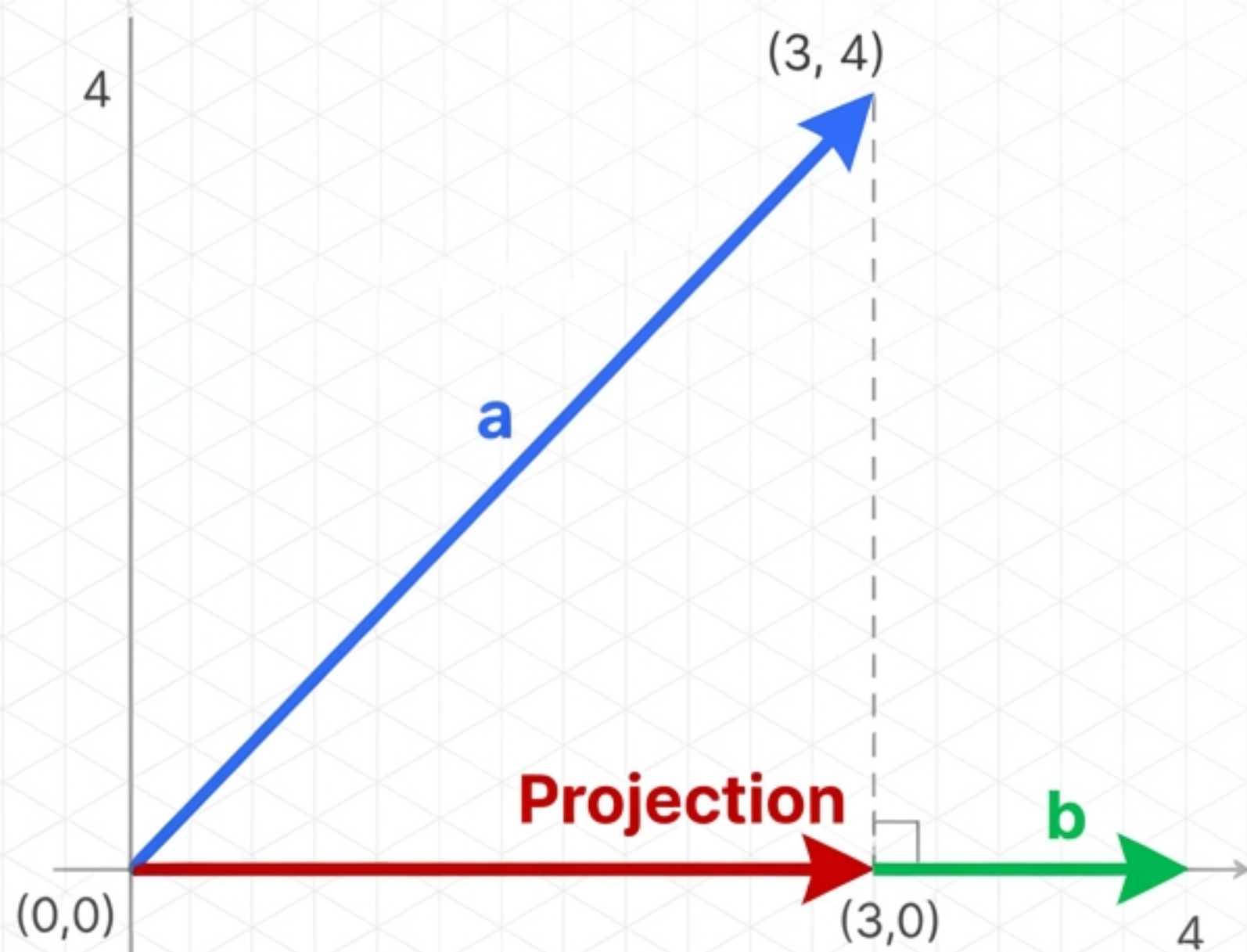
```
import numpy as np
a = np.array([1, 2])
b = np.array([3, 4])

cos_theta = np.dot(a, b) /
    (np.linalg.norm(a) * np.linalg.norm(b))
angle = np.degrees(np.arccos(cos_theta))

print(f'Angle: {angle:.2f}')
```

Angle: 16.26

Vector Projection: Casting a Shadow



Projection measures how much of Vector **A** lies in the direction of Vector **B**.

$$\text{proj}_b(\mathbf{a}) = \frac{(\mathbf{a} \cdot \mathbf{b})}{\|\mathbf{b}\|^2} \cdot \mathbf{b}$$

Dot Product = 12

Magnitude Squared of **b** = 16

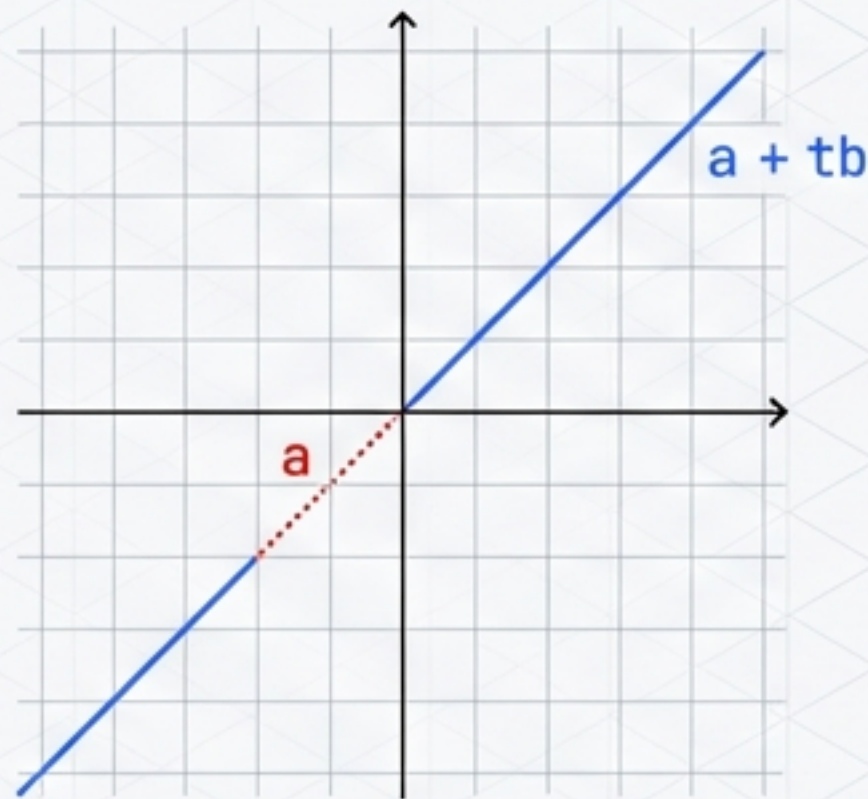
Scalar Factor = $12/16 = 0.75$

Result = $0.75 * (4,0) = (3,0)$

Scaling Dimensions: Line to Hyperplane

1D Line

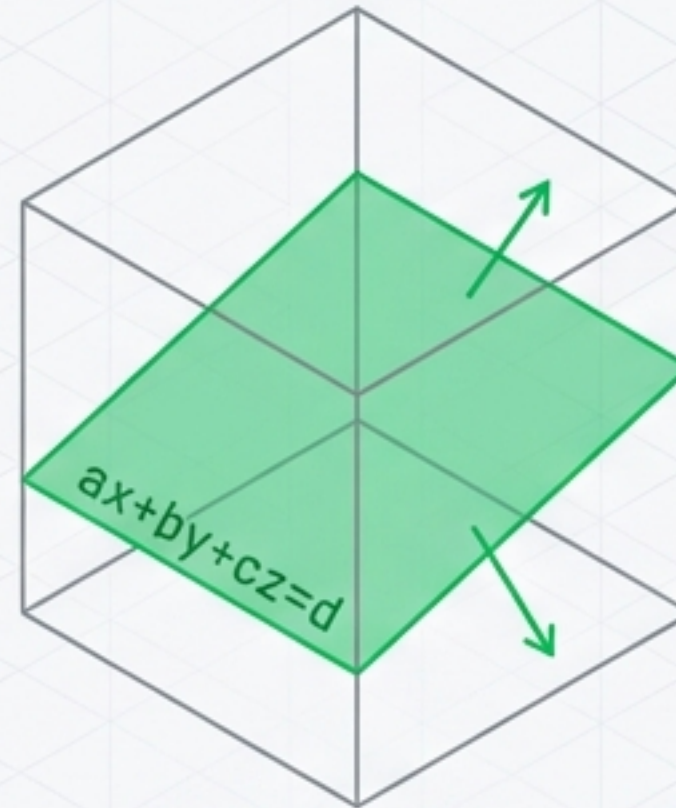
$$r(t) = a + tb$$



Simple Decision Boundary

2D Plane

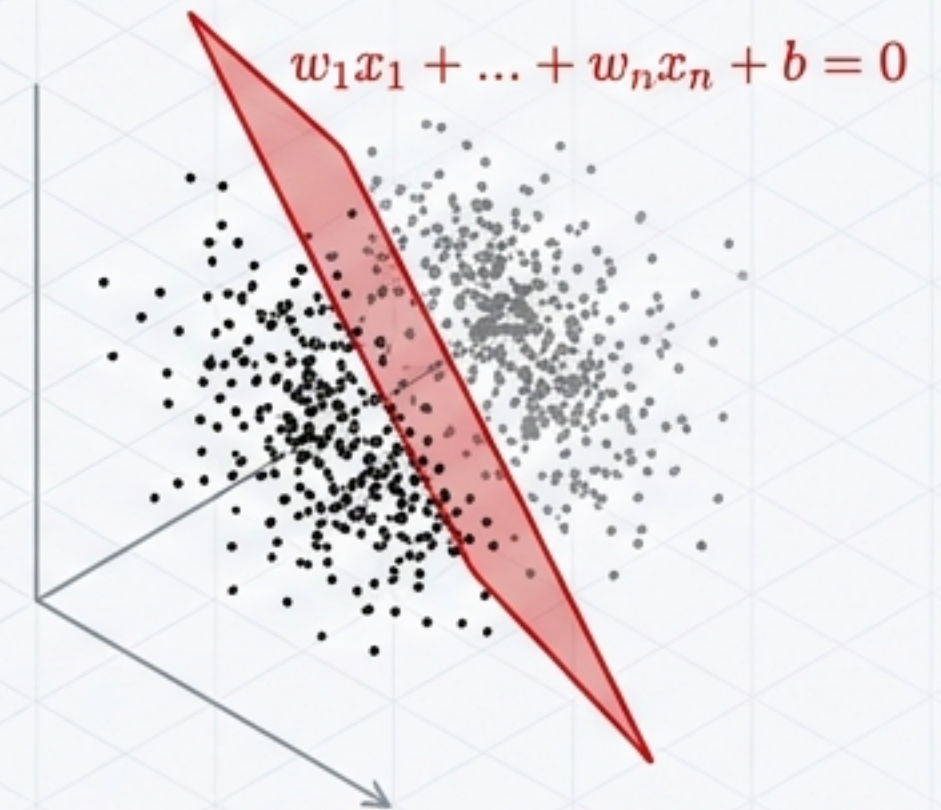
$$ax + by + cz = d$$



Class Separator in 3D

nD Hyperplane

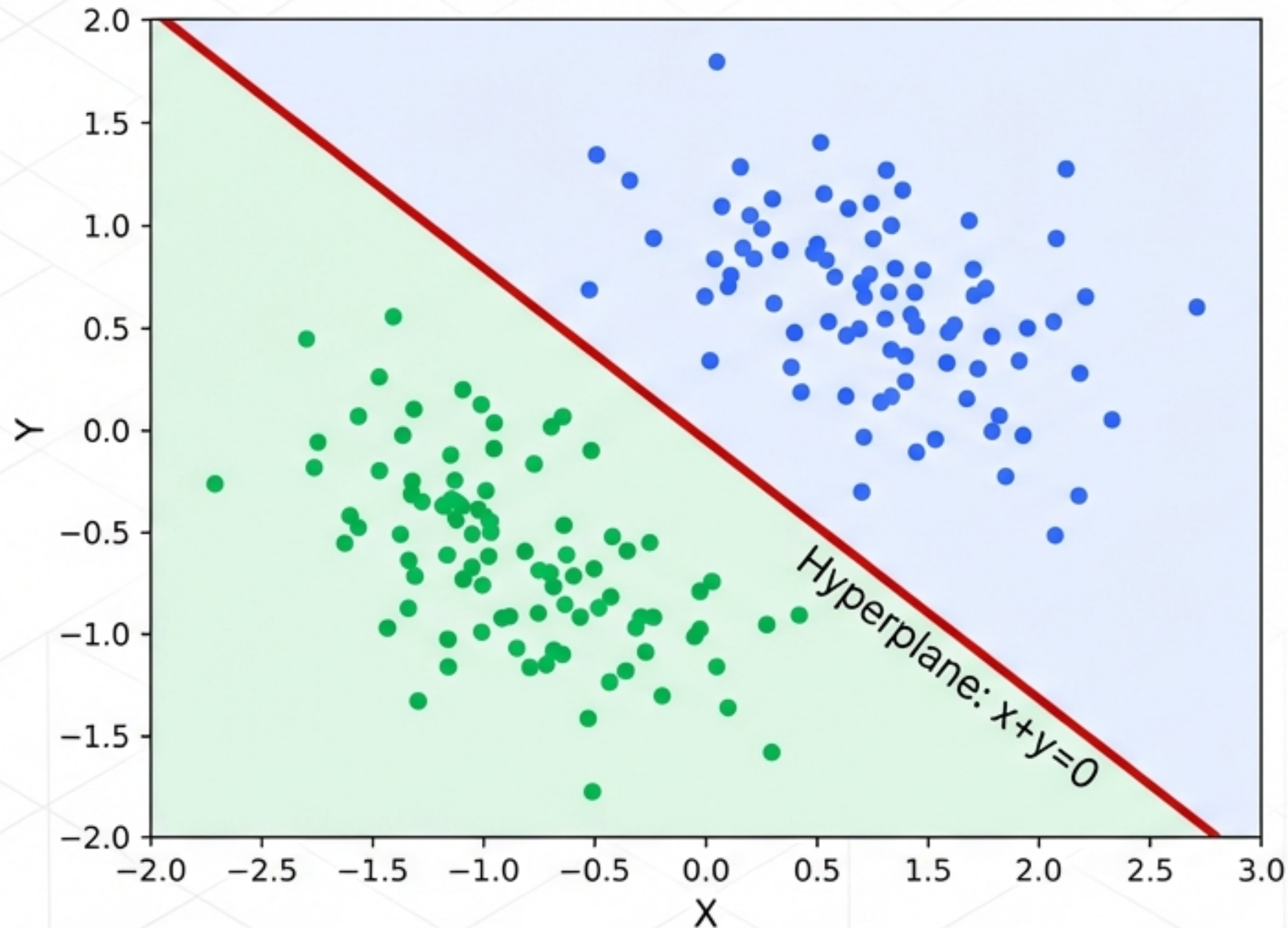
$$w_1x_1 + \dots + w_nx_n + b = 0$$



Generalised Decision Boundary
(n-1 dimensions)

Key Rule: A hyperplane is always one dimension lower than the space it inhabits.

The Hyperplane as a Decision Boundary



- ML Context: Support Vector Machines (SVM).

The hyperplane acts as the decision boundary. The goal is to find the orientation that maximises the margin between the two classes.

Equation Example (4D):

$$x_1 + 2x_2 - x_3 + 4x_4 = 7$$

The Matrix: Data Containers

A rectangular array of numbers arranged in rows and columns ($m \times n$).

Row Matrix ($1 \times n$)

$[1 \ 2 \ 3 \ 4]$

Shape: $1 \times n$

Column Matrix ($m \times 1$)

$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix}$

Shape: $m \times 1$

Square Matrix ($n \times n$)

$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$

Shape: $n \times n$ (rows = columns)

Diagonal Matrix

$\begin{bmatrix} 5 & 0 & 0 \\ 0 & 7 & 0 \\ 0 & 0 & 9 \end{bmatrix}$

Non-zero elements only on the diagonal.

Special Matrices

Identity Matrix (I)

$$I = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The “1” of matrices.

Symmetric Matrix

$$\text{Sym} = \begin{bmatrix} 2 & 3 & 4 \\ 3 & 5 & 6 \\ 4 & 6 & 7 \end{bmatrix}$$

$A = \text{Transpose of } A$

Upper Triangular Matrix

$$U = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 5 & 6 \\ 0 & 0 & 9 \end{bmatrix}$$

Zeros below diagonal

Matrix Arithmetic

Addition & Subtraction

Element-wise operation.
Matrices must be the same order.

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} 4 \\ 6 \end{bmatrix}$$

Scalar Multiplication

Scales every element.

$$2 \times \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

Matrix Multiplication (AB)

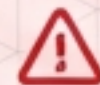
$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$B = \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix}$$

$$(1 * 2 + 2 * 1)$$

Row-by-Column Dive.
Columns of A must
equal Rows of B.

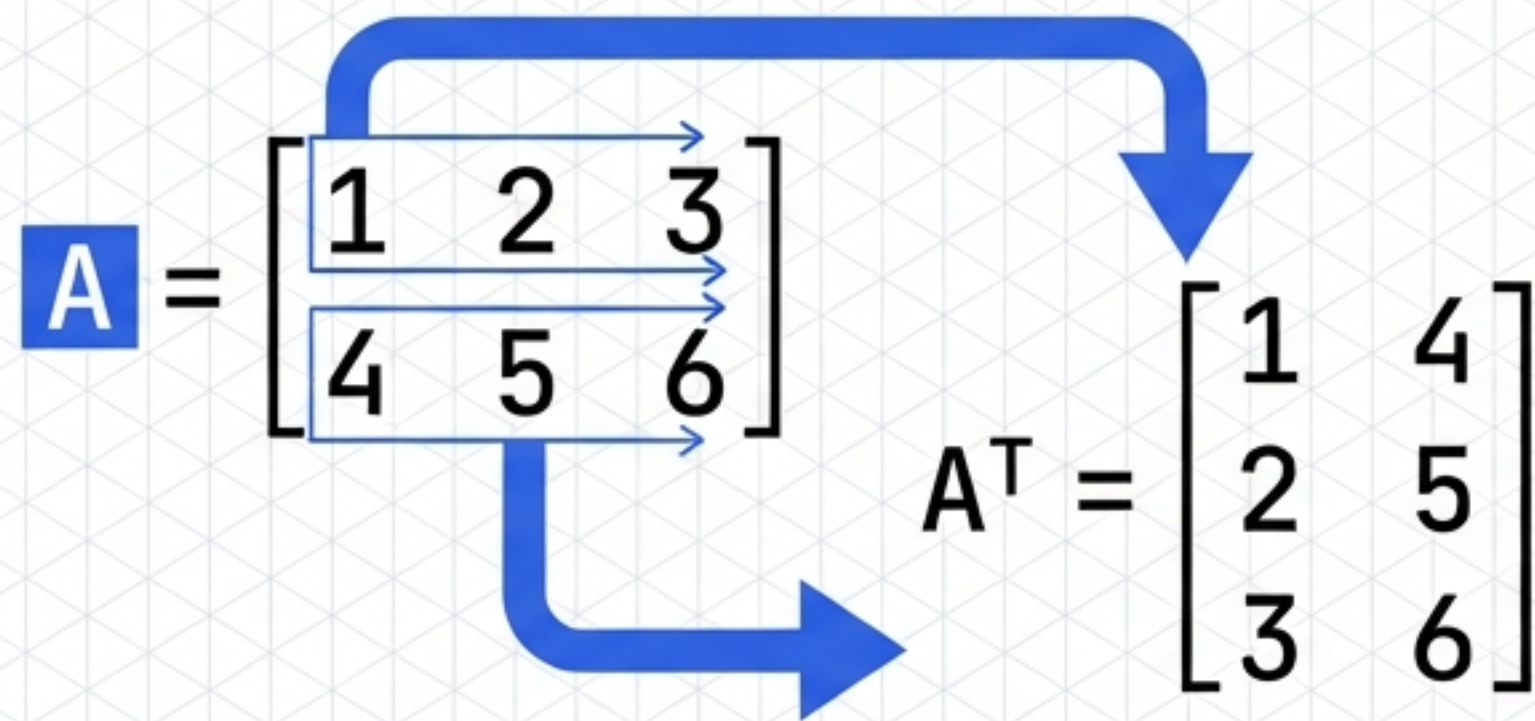
$$AB = \begin{bmatrix} (1 * 2 + 2 * 1) & (1 * 0 + 2 * 2) \\ (3 * 2 + 4 * 1) & (3 * 0 + 4 * 2) \end{bmatrix} = \begin{bmatrix} 4 & 4 \\ 10 & 8 \end{bmatrix}$$



Critical Rule: $AB \neq BA$ (Not Commutative).

Transpose and Determinant

Transpose (A^T)



Flips rows into columns.

Determinant ($|A|$)

$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$

$$|A| = ad - bc$$

A single number representing the scaling factor of the matrix area.

Inverting the Matrix

Swiss Technical Blueprint

The Inverse Formula

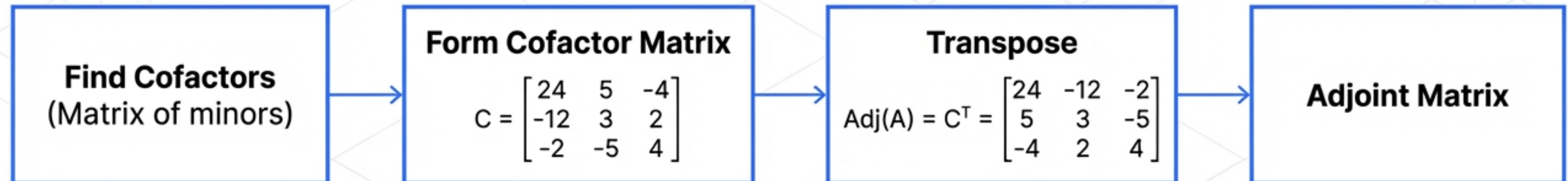
$$A^{-1} = \frac{1}{|A|} \text{Adj}(A)$$

Must not be 0

Singularity

- Singular Matrix: $\text{Det} = 0$ (No Inverse).
- Non-Singular Matrix: $\text{Det} \neq 0$ (Invertible).

The Adjoint Workflow



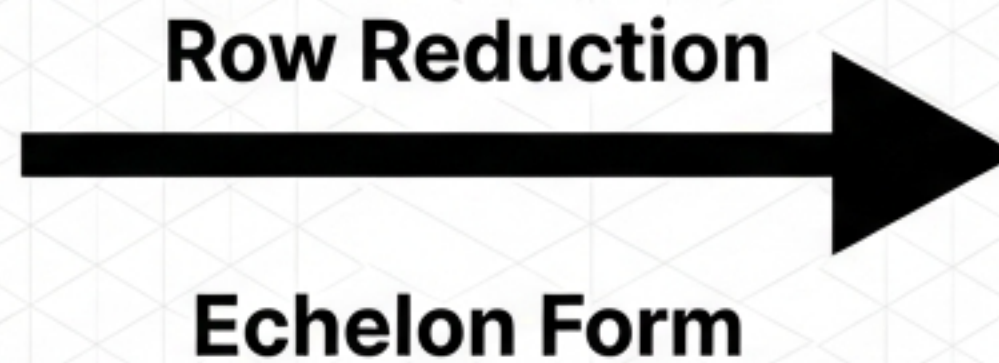
Rank of a Matrix

The measure of information density.

The maximum number of linearly independent rows or columns.

$$\begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 1 & 1 \end{bmatrix}$$

Initial Matrix A



$$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 0 & 0 \\ 0 & 1 & 2 \end{bmatrix}$$

One row eliminated.
Remaining non-zero rows = 2.

Rank = 2

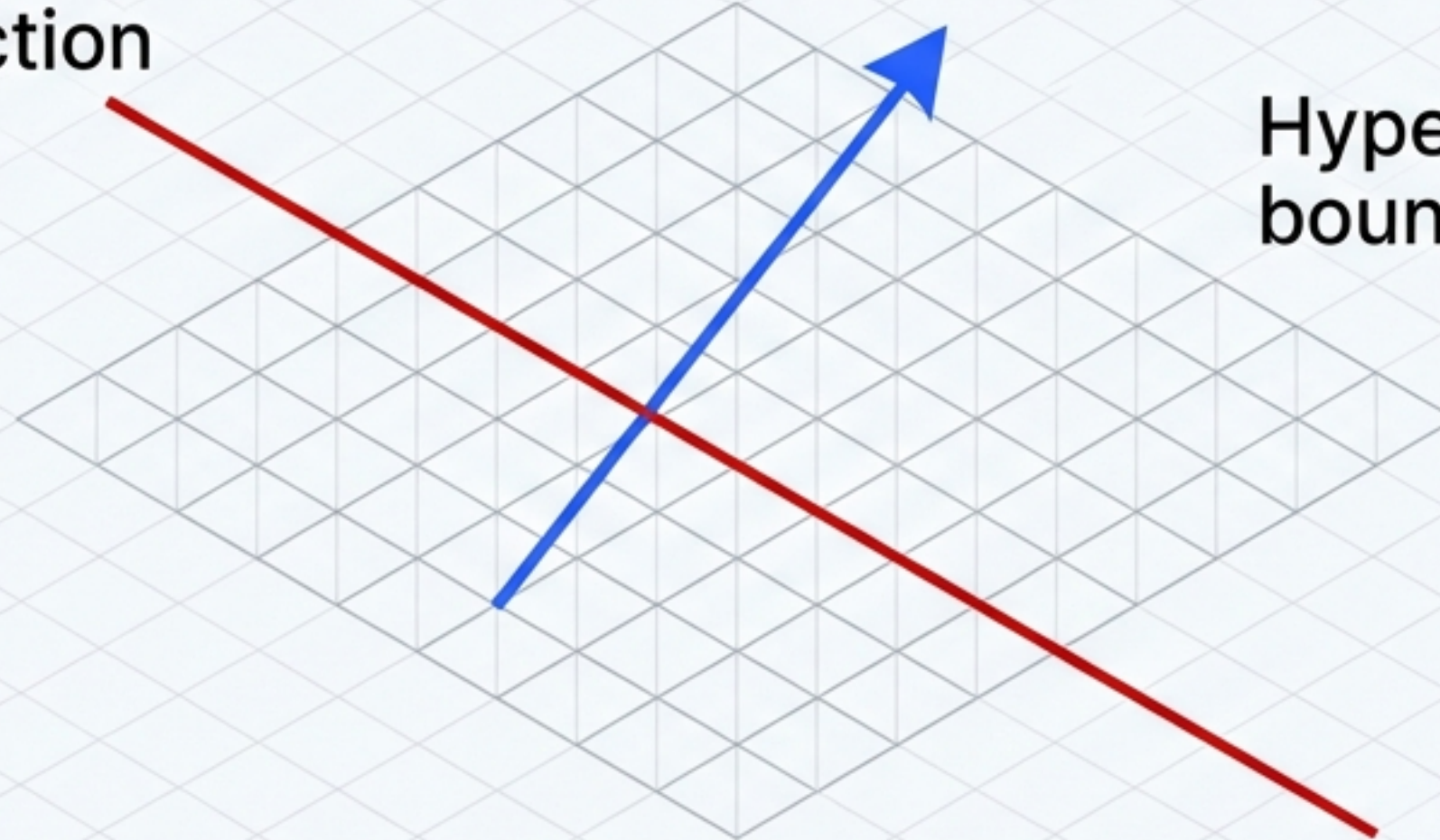
The Python Toolkit (NumPy)

Vector Operations	Matrix Operations
• <code>np.dot(a, b)</code> : Dot product. • <code>np.linalg.norm(a)</code> : Magnitude (Length). • <code>np.arccos(val)</code> : Inverse cosine (radians). • <code>np.degrees(rad)</code> : Convert to degrees.	• <code>np.linalg.det(a)</code> : Determinant. • <code>np.linalg.inv(a)</code> : Inverse. • <code>np.transpose(a)</code> : Transpose (or <code>a.T</code>). • <code>np.linalg.matrix_rank(a)</code> : Matrix Rank.

Summary: The Foundations of Data Space

Vectors define direction and similarity.

Hyperplanes create boundaries for classification.



Matrices store data and drive transformations.

These structures form the mathematical bedrock of modern Machine Learning.